

1 PHP – úvod

Zdroje přednášky

- <https://www.w3schools.com>
- Jiří Kosek, editor. PHP - tvorba interaktivních internetových aplikací -.podrobný průvodce. 1. vyd. Praha : Grada, 1999.
Volně ke stažení: <https://rjwebdesign.cz/blog/tipy-triky/20140826-jiri-kosek-dava-volne-stazeni-sve-knihy-o-html-a-php>

PHP - HypertextPreprocesor

1.1 Syntaxe

```
<?php
// PHP code goes here
?>
```

Soubor má příponu zpravidla .php

[Příklad](#) – vložení php do html

Příklad

```
<!DOCTYPE html>
<html>
<body>

<h1>My first PHP page</h1>

<?php
echo "Hello World!";
?>

</body>
</html>
```

Proměnné in PHP

- The [Variable](#) name is case-sensitive – malá x velká písmena
- Variable name doesn't begin number - \$15.

1.2 Komentáře

// Komentář z jazyka C

1.3 Echo

```
<?php
echo "<h2>PHP is Fun!</h2>";
echo "Hello world!<br>";
echo "I'm about to learn PHP!<br>";
echo "This ", "string ", "was ", "made ", "with multiple
parameters.";
?>
```

S proměnnými

```
?php
$txt1 = "Learn PHP";
$txt2 = "W3Schools.com";
$x = 5;
$y = 4;

echo "<h2>" . $txt1 . "</h2>";
echo "Study PHP at " . $txt2 . "<br>";
echo $x + $y;
?>
```

1.4 Print

```
<?php
print "<h2>PHP is Fun!</h2>";
print "Hello world!<br>";
print "I'm about to learn PHP!";
?>
```

```
<?php
$txt1 = "Learn PHP";
$txt2 = "W3Schools.com";
$x = 5;
$y = 4;
```

```
print "<h2>" . $txt1 . "</h2>";
print "Study PHP at " . $txt2 . "<br>";
print $x + $y;
?>
```

2 Datové typy

PHP obsahuje celkem 6 datových typů:

- *integer* (celá čísla)
- *double* (desetinná čísla)
- řetězce (*string*)
- boolean
- *pole* (*array*)
- *objekty* (*objects*).

Příklad automatické deklarace

```
$a = 15 ; // proměnná typu integer
$b = 15.5 ; // proměnná typu double
$c = "15" ; // proměnná typu string
```

2.1 Typ Integer

od -2 147 483 648 až do + 2 147 483 647.

Po přesáhnutí této hranice PHP automaticky převede proměnnou na typ double.

Zápis

desítkově (základ je 10) - \$decimal = 15 ;
osmičkově (základ je 8) - \$octal = 020 ;
šestnáctkově (základ je 16) - \$hex = 0x10 ;

2.2 Typ Double

od 1,7E-308 do 1,7E+308.

Zápis

```
$a = 0.017 ;
$b = 17.0E-3 ;
```

2.3 Typ String

Typ string (řetězec) se zapisuje do apostrofů nebo také do úvozovek.

Například:

```
$pozdrav = "Ahoj světe !" ;
$pozdrav2 = 'Ahoj světe po druhé !' ;
```

Znaky, které nelze napsat přímo, musíme řešit pomocí escape sekvence:

Sekvence	Význam
\t	Tabulátor
\n	Nový řádek
\r	Návrat vozíku
\\$	Znak dolaru (\$)
\"	Úvozovky (")

2.4 PHP Boolean

A Boolean představuje 2 možnosti: TRUE or FALSE.

```
$x = true;  
$y = false;
```

2.5 PHP Object

Objekt je datový typ, který uchovává data a informace o tom, jak tyto údaje zpracovávat. V PHP musí být objekt explicitně deklarován.

Nejprve musíme deklarovat třídu objektu. K tomu používáme klíčové slovo třídy. Třída je struktura, která může obsahovat vlastnosti a metody:

příklad

```
<?php  
class Car {  
    function Car() {  
        $this->model = "VW";  
    }  
}  
  
// create an object  
$herbie = new Car();  
  
// show object properties  
echo $herbie->model;  
?>
```

2.6 Pole

```
$a[0]= "jabka";  
$a[1]= "hrusky";
```

Jiná možnost inicializace:

```
$a[ ]= "jabka"; // zde je index 0  
$a[ ]= "hrusky"; // zde je index 1  
nebo $a=Array("jabka","hrusky","svestky");
```

Inicializace od jiného indexu

```
$dny = Array(1=>"Po","Ut","St","Ct","Pa","So","Ne"); // Po ma index 1
```

Místo číselného indexu pole je možné použít i typ string – **asociativní pole**:

```
$adresy["Jirka"]=strnad@seznam.cz  
echo $adresy["Jirka"]; // vypíše se strnad@seznam.cz
```

2.6.1 Asociativní pole pro převod

```
$day2den = Array("Mon"=>"Po", "Tue"=>"Ut")  
pak  
echo $day2den["Tue"]; vypíše Ut
```

2.6.2 Funkce pro práci s polem

Count – zjištění počtu prvků v poli

Reset – přesun ukazatele na 1. prvek

Current – hodnota prvku na který ukazuje ukazatel

Next – nejprve se ukazatel přesune a pak vrátí hodnotu

Key – vrací index prvku pole, na který ukazuje ukazatel

list – načtení prvků pole do proměnných

Příklad

```
$x=Array("Po","Ut","St");  
list ($p1,$p2,$p3) = $x;
```

2.6.3 Vícerozměrná pole

`$mat[3][5]` – chová se jako jednorozměrné pole, jehož prvky jsou pole

Příklad `$mat[3]` – vrací pole, které obsahuje všechny prvky 3. řádku

2.7 PHP NULL Value

Null je speciální datový typ, který může mít pouze jednu hodnotu: NULL.

Proměnná datového typu NULL je proměnná, která nemá žádnou hodnotu přiřazenou.

Tip: Pokud je proměnná vytvořena bez hodnoty, automaticky se přiřadí hodnota NULL.

Proměnné lze také vyprázdnit nastavením hodnoty na hodnotu NULL:

Příklad

```
<?php
$x = "Hello world!";
$x = null;
var_dump($x);
?>
```

3 PHP String Functions

Přístup k jednotlivým znakům řetězce

```
$text="ukázkový text"
```

```
echo $text ; //vypíš se celý text
```

```
echo $text[2]; // vypíše písmeno á
```

3.1 Get The Length of a String

PHP `strlen()` function vrací délku řetězce .

Příklad

```
<?php
echo strlen("Hello world!"); // outputs 12
?>
```

3.2 Počet slov

PHP `str_word_count()` vrací počet slov v řetězci:

Příklad

```
<?php
echo str_word_count("Hello world!"); // outputs 2
?>
```

3.3 Reverse String

`strrev()` function provede reverzi řetězce:

Příklad

```
<?php
echo strrev("Hello world!"); // výstup !dlrow olleH
?>
```

3.4 Search v řetězci

PHP `strpos()` funkce hledá text v řetězci.

Příklad

```
<?php
echo strpos("Hello world!", "world"); // výstup 6
?>
```

3.5 Replace

`str_replace()` funkce nahradí text v řetězci

Příklad

```
<?php
echo str_replace("world", "Dolly", "Hello world!"); //výstup Hello
Dolly!
?>
```

3.6 Konstanty

Vytvoření konstanty se provede funkcí `Define()`. Funkce má tři parametry. První parametr obsahuje jméno konstanty. Druhý parametr obsahuje hodnotu konstanty. Třetí parametr je nepovinný a určuje, zda bude jméno konstanty závislé na velikosti písmen. Standardně jsou konstanty citlivé na velikost písmen. Chceme-li se na konstantu odkazovat různě velkými písmeny, nastavíme třetí parametr jako `true`.

```
Define ("Pi", 3.14);
```

```
Define ("MIN_HODNOTA", 0);
```

3.7 Přetypování proměnných

(double) + (jiný typ se převede na double) = (double)

Příklad:

```
$i=10; // i je integer
```

```
$x=(double)$i // x je double
```

GetType(proměnná) – zjištění typu proměnných

SetType(proměnná , typ) – změna typu proměnné

Příklad: `SetType($N,"string")`

3.8 Konverze řetězců na čísla

Výsledkem je double nebo integer.

<code>\$k = 1 + "13.5"</code>	14.5	double
<code>\$k = 1 + "-1.3e3"</code>	-1299	double (je zde e)
<code>\$k = 1 + "pes3"</code>	4	double (je zde e)
<code>\$k = 1 + "20BLECH"</code>	21	double (je zde E)
<code>\$k = 1 + "20psu"</code>	21	integer

3.9 Výrazy

Jsou vyhodnocovány z prava do leva.

```
$a=0; $b=0;
```

```
$b=($a=0);
```



```
$b=$a=0;
```

3.10 Matematické operátory

+, -, *, /, %, - zbytek po celočíselném dělení

```
$x = $x + <<výraz>>      $x += <<výraz>>
```

+=, -=, *=, /=, %=

++ inkrementace \$i++ \$i=\$i+1

-- dekrementace \$i-- \$i=\$i-1

++\$i, --\$i operace se provede před vrácením hodnoty

3.11 Logické výrazy a operátory

Pozor – v PHP jsou záporná čísla pravdivý výraz.

= = rovná se

!= <> nerovná se

<, > větší, menší

&& logický součin AND

|| logický součet OR

XOR exkluzivní or

! negace

3.12 Spojování řetězců

Spojování řetězců se provádí znakem „.“

```
echo "vysledek je".(10/3);
```

```
$a=$x. "a dalsi text";
```

3.13 Podmíněný operátor

```
<<výraz>>?<<výraz2>>:<<výraz3>>
```

Příklad \$min = \$x<\$y ? \$x : \$y; // do min uloží x nebo y

3.14 Priorita operátorů

index pole
! ~ -- ++ přetypování @
* / %
+ - .
<< >>
< <= >= >
== != <>
&
^

|
 &&
 ||
 ?:
 = + - . = * = / = , = % = & = | = ^ = > > = < < =
 AND
 XOR
 OR
 ,

3.15 Příkazy pro větvení

3.15.1 příkaz if

syntaxe

if (<<výraz>>) <<příkaz>>;

if (\$b != 0) \$podíl=\$a/\$b;

if (<<výraz>>):	nebo	if (<<výraz>>)
<<příkaz1>>;		{
<<příkaz2>>;		<<příkaz1>>;
endif;		<<příkaz2>>;
if (<<výraz>>)		}

3.15.2 příkaz if-else

```

if (<<výraz>>)
{
<<příkaz1>>;
}
else
{
<<příkaz2>>;
}

```

```

if (<<výraz>>):
<<příkaz1>>;
<<příkaz2>>;
else:
<<příkaz1>>;
<<příkaz2>>;
endif;

```

3.15.3 if-elseif-else

if (<<výraz1>>)

```

    <<příkaz1>>;
elseif (<<výraz2>>)
    <<příkaz2>>;
elseif (<<výraz3>>)
    <<příkaz3>>;
else
    <<příkazN>>;

```

Příklad

```

<?php
$t = date("H");

if ($t < "10") {
    echo "Have a good morning!";
} elseif ($t < "20") {
    echo "Have a good day!";
} else {
    echo "Have a good night!";
}
?>

```

3.15.4 switch

```

switch (<<výraz>>)
{
case <<hodnota1>>:
    <<příkaz1_1>>;
    <<příkaz1_2>>;
    ...
    break;
case <<hodnota2>>:
    <<příkaz2_1>>;
    <<příkaz2_2>>;
    ...
    break;
...
default
    <<příkazX_1>>;
    <<příkazX_2>>;
    ...
}

```

Příklad

```

<?php
$favcolor = "red";

switch ($favcolor) {
    case "red":
        echo "Your favorite color is red!";
        break;
    case "blue":
        echo "Your favorite color is blue!";
        break;
    case "green":
        echo "Your favorite color is green!";
        break;
    default:
        echo "Your favorite color is neither red, blue, nor
green!";
}
?>

```

3.16 Cykly

3.16.1 while

while (<<výraz>>)<<příkaz>>;

Příklad

```

$i = 0;
while ($i<100)
{
    echo "$^2".($i*$i). "<BR>";
    $i++;
}

```

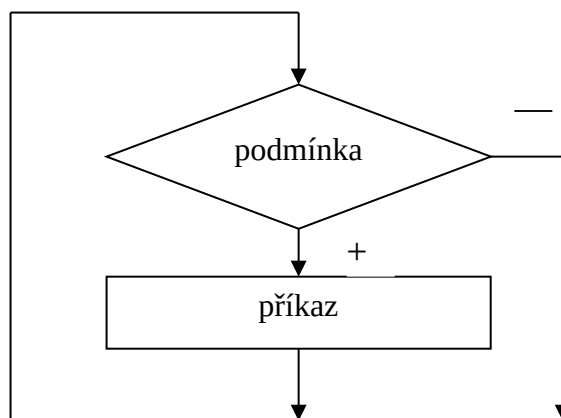
nebo

```

$i = 0;
while ($i<100):

    echo "$^2".($i*$i). "<BR>";
    $i++;
endwhile;

```

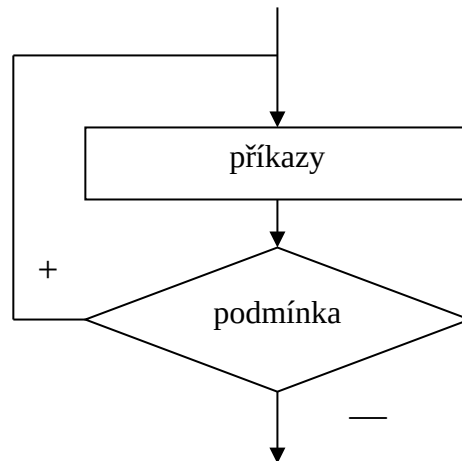


3.16.2 do-while

```
do
<<příkaz>>
while (<<výraz>>);
```

Příklad

```
<?php
$i = 0;
do
{
echo "$^2".($i*$i). "<BR>";
$i++;
}
while ($i<100);
?>
```



3.16.3 for

```
for (<<výraz1>>;<<výraz1>>;<<výraz1>>)<<příkaz>>;
```

Příklad

```
for ($i =0; $i<100; $i++); echo "$^2".($i*$i). "<BR>";
```

Pro více příkazů se opět použijí {,}.

3.17 PHP foreach Loop

Vypíše obsah pole, objektu,...

```
<?php
$colors = array("red", "green", "blue", "yellow");

foreach ($colors as $barva) {
    echo "$barva <br>";
}
?>
```

3.18 Příkaz Break

Ukončuje provádění cyklu.

3.19 Continue

Přeskočí příkazy ve zbývajících částech těla cyklu a začne provádět další iteraci.

3.20 Vložení kódu z jiného skriptového souboru

3.20.1 Require

Require slouží k načtení skriptu ze souboru. Skript se vloží v místě použití příkazu.

Příklad

```
funkce.php
<?php
define("bydliste","Jihlava"); //definice konstanty
echo bydliste;
?>
```

```
start.php
<?php
require("funkce.php");
echo bydliste;
?>
```

Příklad databáze

```
connect.php
<?php
$servername = "localhost";
$username = "smrcka";
$password = "heslo";
$dbname = "smrcka";
$spojeni = mysqli_connect($servername, $username, $password,
$dbname);
mysqli_set_charset($spojeni, "utf8");
?>
```

```
delete.php
<?php
require("connect.php");

$sql = "DELETE FROM tabulka WHERE cislo = '$_GET[oc]'";
if (mysqli_query($spojeni, $sql)) {
    echo "Záznam byl úspěšně smazán";
} else {
    echo "Chyba při mazání: " . mysqli_error($spojeni);
}
mysqli_close($spojeni);
?>
```


Prohlížení všech zaměstnanců

3.20.2 Include

Tento příkaz slouží k načtení skriptu ze souboru. Načítaný skript se vloží v místě použití příkazu. Na rozdíl od require se tento příkaz může volat několikrát – například uvnitř cyklu.

Příklad

```
<?php
for ($i=1;$i<5;$i++):
include "../funkce.php";
endfor;
?>
```

```
<?php
include_once "a.php"; // this will include a.php
include_once "A.php"; // this will include a.php again! (PHP 4 only)
?>
```

3.20.3 include_once

Pokud kód ze souboru již byl zahrnut, nebude znovu zahrnut a include_once vrátí TRUE. Jak název napovídá, soubor bude zahrnut pouze jednou.

3.21 Funkce

Definice

```
function <<jméno_funkce>>(<<parametr1>>,<<parametr2>>,...)
{
    příkazy
}
```

Příklad

```
function pozdrav( )
{
    echo " zdravíme vás"
}
```


Příklad

```
<?php
function faktorial($n)
{
    for ($i=$n;$n>0;$n--) $i*=$n;
    return $i;
}
for ($k=1; $k<=100 ; $k++) echo "$k!=".faktorial($k)."<BR>"
;
?>
```

Příklad

```
<?php

function max($a,$b)
{
    return $a>$b?$a:$b;
}
echo max(10,12);

?>
```

3.21.1 Rozsah platnosti proměnných

global – zpřístupní globální proměnnou uvnitř funkce

`$GLOBALS[nazev_promenne]`

```
<?php
$x = 75;
$y = 25;

function addition() {
    $GLOBALS['z'] = $GLOBALS['x'] + $GLOBALS['y'];
}

addition();
echo $z;
?>
```

3.21.2 Předávání parametru odkazem &

Příklad `function test(&$a,$b)`

3.21.3 Nastavení standardních hodnot parametrů

`function umocni($zaklad , $exponent=2)`

3.21.4 Statické proměnné

Statická proměnná je inicializovaná pouze při definici proměnné, při dalších voláních již ne. Vytváří se příkazem **static**.

```
function citac()
{
    $n=0;
    $n++;
    return $n;
    // výsledek po čtyřnásobném spuštění 1 1 1 1
}

{
    static $n=0;
    $n++;
    return $n;
    // výsledek po čtyřnásobném spuštění 1 2 3 4
}
```

3.21.5 Rekurze

Příklad

```
n!=n*(n-1)!
function faktorial($a)
{
    return $a==0 ? 1:$a*faktorial($a-1);
}
```

3.22 Příkaz eval

Tento příkaz převede libovolný parametr (řetězec) na příkaz PHP.

3.23 Předdefinované proměnné

Vedle proměnných, které jsou ve skriptu definované uživatelem, existují i předdefinované proměnné. Často se tyto proměnné používají pro načtení hodnot, předaných do skriptu v rámci URL adresy (metoda GET protokolu HTTP) nebo při předání hodnot z formuláře (například metoda POST protokolu HTTP).

3.24 Seznam předdefinovaných proměnných:

- **\$GLOBALS** – odkaz na každou proměnnou, která je dostupná ve skriptu.
- **[\\$_SERVER](#)** – proměnné nastavované WWW-serverem. Nahrazuje stále platnou předdefinovanou proměnnou `$HTTP_SERVER_VARS`.
- **[\\$_GET](#)** – proměnné poskytované skriptu metodou GET protokolu HTTP. Nahrazuje stále platnou předdefinovanou proměnnou `$HTTP_GET_VARS`.
- **[\\$_POST](#)** – proměnné poskytované skriptu metodou POST protokolu HTTP. Nahrazuje stále platnou předdefinovanou proměnnou `$HTTP_POST_VARS`.
- **[\\$_COOKIE](#)** – proměnné poskytované skriptu přes cookies. Nahrazuje stále platnou předdefinovanou proměnnou `$HTTP_COOKIE_VARS`.
- **[\\$_FILES](#)** – proměnné poskytované skriptu HTTP protokolem při přenosu (*upload*) souboru. Nahrazuje stále platnou předdefinovanou proměnnou `$HTTP_POST_FILES`.
- **[\\$_ENV](#)** – proměnné poskytované skriptu z prostředí, ve kterém běží PHP. Nahrazuje stále platnou předdefinovanou proměnnou `$HTTP_ENV_VARS`.
- **[\\$_REQUEST](#)** – proměnné poskytované skriptu z libovolného vstupu.
- **[\\$_SESSION](#)** – proměnné, které jsou registrovány pro aktuální relaci (*session*) skriptu. Nahrazuje stále platnou předdefinovanou proměnnou `$HTTP_SESSION_VARS`.

4 FORMULÁŘE

<FORM ACTION = skript, který zpracuje data z formuláře method= metoda přenášení dat>

Data z formuláře lze poslat i pomocí e-mailu: action=mailto:smrcka@neco.cz“.

4.1 Metoda přenášení dat

get – předává data přímo v URL. Všechna přenášená data jsou v URL stránky vidět.

post – předává data v hlavičce http požadavku.

4.2 Příklad

form1.php

```
<form action = "zpracuj.php" method="get">
<input type="text" name="cislo1" ><br>
<input type="text" name="cislo2" ><br>
<input type="submit" value="odesli" ><br>
</form>
```

zpracuj.php

```
<?php
echo $cislo1."<BR>";
echo $cislo2."<BR>";
?>
```

Globals = off v php.ini

```
<?php
echo $_GET[cislo1]."<BR>";
echo $_GET[cislo2]."<BR>";
?>
```

4.3 Tlačítko pro odeslání formuláře

form2.php

```
<form action="smlouva.php" method="get" >

potvrďte prosím smlouvu

<input type="submit" name="potvrzeni" size=30 value="ano" >

<input type="submit" name="potvrzeni" size=30 value="ne" >

</form>
```

Soubor smlouva.php pak vypadá následovně:

```
<?php
if ($_GET[Potvrzeni]=="Ano") echo "děkuji za potvrzení" ;
else echo "litujeme, že nesouhlasíte se smlouvou"
?>
```

4.4 Heslo

```
<INPUT TYPE=PASSWORD NAME="heslo" method="POST">

<form action="heslo.php" method="post" >

<input type="password" name="heslo">zadejte prosím heslo

<input type=submit value="odesli">
```

Soubor heslo.php pak vypadá následovně:

```
<?php
if ($_POST[heslo]=="listopad") echo "heslo je správné" ;
else echo "chybné heslo, zadejte znovu";
?>
```

4.5 Zaškrťovací pole

TYPE=CHECKBOX

kava.htm

```
<form action="kava.php" method="post" >

do kávy si dáte <br>

<input type="checkbox" name="cukr" value="ano" >cukr

<input type="checkbox" name="mleko" value="ano" >mléko

<input type="submit" value="objednávka">

</form>
```

Soubor kava.php pak vypadá následovně:

```
<?php
if ($_POST[cukr]=="ano") echo "oslaď si kávu "."<BR>" ;
```

```
if ($_POST[mleko]=="ano") echo "dej si mléko " ;
?>
```

Použití pole ve formuláři

```
<form action="pocitac.php" method="post" >
<input type="checkbox" name="konf[]" value="cd-rom" >mechanika
cd-rom
<input type="checkbox" name="konf[]" value="zip" >mechanika zip
100mb
<input type="checkbox" name="konf[]" value="sound" >zvuková karta
<input type="checkbox" name="konf[]" value="modem" checked
>modem
<input type="submit" value="potvrzení konfigurace" >
</form>
</form>
```

Soubor pro zpracování tohoto formuláře pak vypadá následovně:

```
<?php
for ($i=0; $i<Count($konf); $i++):
    if ($konf[$i]=="cd-rom"): echo "namontuj CR-ROM "."<BR>" ;
    elseif ($konf[$i]=="zip"): echo "namontuj Zipku "."<BR>" ;
    elseif ($konf[$i]=="sound"): echo "namotuj zvukovou kartu
"."<BR>" ;
    elseif ($konf[$i]=="modem"): echo "namontuj modem "."<BR>" ;
    endif;
endfor;
?>
```

Poznámka Count(\$konf) – zjistí počet prvků pole.

4.6 Zaškrtávací pole

TYPE=RADIO

```
<form action="str100.php" method="post" >
<blockquote>
<input type="radio" name="disk" value=850     > disk 850 mb
<input type="radio" name="disk" value=1200     > disk 1200 mb
<input type="radio" name="disk" value=1600     > disk 1,6 gb
<input type="radio" name="disk" value=2100     > disk 2,1 mb
</blockquote>
<input type="submit" value="potvrzení konfigurace" >
</form>
```

Soubor pro zpracování tohoto formuláře pak vypadá následovně:

```
Přejete si disk o velikosti:  
<?php  
    echo $_POST[disk]." MB"; ?>
```

4.7 Tlačítko s obrázkem

```
<INPUT TYPE=IMAGE>
```

mapa.htm

```
<form method="post" action="mapa.php" >  
vyberte si stát:<br>  
<input type="image" name="mapa" src="mapa.jpg" alt="nějaká mapa">  
</form>
```

Po odeslání tohoto formuláře se v proměnných přenesou aktuální souřadnice ukazatele myši.
(název_souboru_x, název_souboru_y).

```
<?php  
echo "souřadnice x " .$_POST['mapa_x']."<BR>";  
echo "souřadnice y " .$_POST['mapa_y']."<BR>";  
?>
```

4.8 Odeslání souboru

<INPUT TYPE=FILE>

Soubor se odesílá na server do příslušného adresáře. Do pole lze zapsat přímo název souboru. Atribut ENCTYPE určuje způsob kódování dat při přenosu. Standardní hodnota je: **application/x-www-form-urlencoded**. Toto kódování je pro soubor nepoužitelné, proto se používá modernější způsob **multipart/form-data**. Pro toto kódování je nutné použít metodu **POST**.

Příklad

upload.php

Soubor pro zpracování tohoto formuláře pak vypadá následovně.

```
<html>
```

```
<head>
```

```
</head>
```

```
<body>
```

```
<form action="#" method="post" enctype="multipart/form-data">
```

```
<input type="file" name="fupload">
```

```
<input type="submit" value="Nahrát">
```

```
</form>
```

```
<?php
```

```
if (isset($_FILES['fupload']))
```

```
{
```

```
    $slozka = "soubory";
```

```
    $cil = $slozka . "/" . $_FILES['fupload']['name'];
```

```
    $nazev_souboru = $_FILES['fupload']['tmp_name'];
```

```
    $copy = move_uploaded_file($nazev_souboru, $cil)
```

```
        or die ("Načtený soubor nelze zkopírovat");
```

```
    //chmod ($cil, 0644);
```

```
    if($copy == true){
```

```
        echo "Soubor " . $_FILES['fupload']['name'] . " byl úspěšně  
nakopírován na server.<br><br><a href=\"\$cil\">Spust' soubor</a>";
```

```
    }else{
```



```

        echo "Soubor " .$_FILES['fupload']['name']. " nemohl být
nakopírován na server.\nP počet chyb: " . $_FILES['fupload']
['error'];
    }
}
?>

```

</body>

</html>Nezapomeňte vytvořit na serveru adresář soubory a nastavit právo w pro ostatní.

4.9 Skrytá pole

<INPUT TYPE="HIDDEN">

Toto pole se ve formuláři vůbec neobjeví.

Příklad

```

hidden.php
<form action = "zpracuj.php" method="post">

<?php $odesli="ano" ?>

<input type="hidden" name="cislo" value="<?php echo $odesli ?> ">
<br>
<input type="submit" value="odesli" ><br>
</form>
zpracuj.php
<?php
echo $cislo1."<BR>";
?>

```

4.10 Seznamy

<SELECT>...</SELECT>

SIZE – počet viditelných prvků

MULTIPLE – možnost výběru více prvků současně

Příklad

```

seznamy.html
<head>
<title></title>
</head>
<body>
<form action="seznamy.php" method="post" >

```

```

vyberte si čísla o která máte zájem
<blockquote>
<select name="cisla" size=1>
<option value="1">1/96
<option value="2">2/96
<option value="3">3/96
<option value="4">4/96
</select>
</blockquote>
vyberte si formát, ve kterém chcete zvolená čísla zaslat
<blockquote>
<select name="format" size=1 >
<option value="html">html
<option value="pdf" selected>pdf
<option value="ps">ps
<option value="ascii">obyčejný text
</select>
</blockquote>
<input type=submit value=odesli >
</form>
</body>
</html>

<?php
echo $_POST[cisla]."<BR>";
?>

```

4.11 Víceřádkový text

<TEXTAREA>... </TEXTAREA>

Příklad

textarea.html

```

<form action="komentar.php" method="post" >
<textarea name="komentar" rows=5 cols=40 >
zde prosím napište připomínky k našemu programu
</textarea>
<input type="submit" value="odesli" >
</form>

```

zpracování se provede:

děkujeme za komentář ve znění

<blockquote>

```
<?php
```

```
echo nl2br($_POST[komentar]) ;
```

```
?>
```

Funkce NL2BR zajistí převedení konce řádků v řetězci na tag

5 Regulární výrazy

Používají se pro kontrolu nebo hledání obsahu řetězců. V následující tabulce jsou uvedeny třídy znaků použitelných v regulárních výrazech.

Výraz	Znamená	Vyhovují	Nevyhovují
abc	Řetězec, který obsahuje tento podřetězec	abc, aascvabcnmn,abcd	abbbvvvc, cba,
^ ^abc	Začátek řetězce	abcd,abc,	bbabc, nnabc
\.	tečka		
\\	lomítko		
* a*b	výraz se může opakovat, nebo nemusí být	aaaabbbb,ab,agb	aaaaddd, a
+	výraz se musí objevit alespoň jednou		
.	libovolný znak		
?	výraz není, nebo je jedenkrát		
\$ a\$	konec řetězce	hasghaa,a	aafdgd, cc, an
a{2,6}	minimální a maximální počet opakování a	aa,aaa,aaaa,aaaaa,aaaaaa	a,aaaaaaa
a{5,}	a je minimálně 5 krát		
a{5}	a je maximálně 5 krát		
[abcd]	jeden znak z množiny		
[a-d]	zkrácený výpis předchozího řádku		
[a-z]	malé písmena		
[A-Z]	velké písmeno		
[a-zA-Z]	malé nebo velké písmeno		
[^ab]	regulární výraz vyhoví libovolnému znaku, kromě těch		

	v závorce		
 Po Ut St Ct Pa So Ne	rozdělení reg. výrazu na podvýrazy řetězec vyhovuje, pokud vyhovuje alespoň jedné části	Po, Ut, St, Ct, Pa, So, Ne	Pon, So, Adhg,12
(Po Ut St Ct Pa So Ne) (Mo Th We Th Fr Sa Su)		Po,St, Mo,.....	
[[:<:]]Novak[[:>:]]	vyhoví pouze řetězec Novak	Novak	Novakova, Novacek,

alnum	Alfanumerické znaky (písmena a číslice)
alpha	písmena anglické abecedy
blank	mezera a tabulátor
cntrl	řídící znaky
digit	Číslice
graph	všechny znaky, které mají grafické znázornění
lower	malá písmena anglické abecedy
print	tisknutelné znaky
punct	interpunkční a další znaky (závorky, zavináče,...)
space	mezera, tab, nová řádka, nová stránka,...
upper	Velká písmena anglické abecedy
xdigit	Šestnáctkové číslice (0-9, a-f, A-F)
[[:<:]][[:digit:]] {4} [[:>:]]	Řetězec obsahující 4 místné číslo

5.1 Použití regulárního výrazu

<?

```
$neco = "cde";
if (EReg("[ab]", $neco, $Cast))
    echo "reg. vyraz je spravny";
else echo "reg vyraz spatny";
```

?>